

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NEHEMIA PANDU INDRAGIRI

109082500019

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Algoritma pencarian berfungsi untuk menemukan keberadaan atau lokasi suatu data spesifik di dalam array. Hasil pencarian yang optimal tidak hanya memberikan status ketemu atau tidak, melainkan mengembalikan nilai posisi indeks dari data tersebut agar dapat diproses lebih lanjut oleh program.

Sequential Search bekerja dengan cara memeriksa setiap elemen array satu per satu secara berurutan dari indeks awal hingga akhir. Prosesnya akan langsung dihentikan (break) seketika data yang dicari sudah ditemukan. Keunggulan metode ini adalah dapat digunakan pada data yang acak maupun terurut, namun kurang efisien untuk data berjumlah besar.

Binary Search merupakan metode pencarian efisien yang memiliki syarat mutlak di mana data di dalam array harus sudah dalam keadaan terurut (membesar atau mengecil). Algoritma ini bekerja dengan membagi dua ruang pencarian secara terus-menerus menggunakan penunjuk indeks *kiri*, *kanan*, dan *tengah* hingga data ditemukan, sehingga sangat cepat untuk menangani data berskala masif.

B. Guided

1. Sequential Search

```
package main

import "fmt"

func SequentialSearch(arrBuah [5] string, dataDicari
string) int {
    var idx_found = -1
    for i:=0; i < len(arrBuah); i++ {
        if arrBuah[i] == dataDicari {
            idx_found = i
            break
        }
    }
    return idx_found
}

func main(){
    var arrBuah [5] string

    for i := 0; i < len(arrBuah); i++ {
        fmt.Printf("Masukan data buah indeks ke - %d
: ", i)
        fmt.Scan(&arrBuah[i])
    }

    var dataCari string
```

```

        fmt.Println("Masukan data buah yang ingin dicari:
")
        fmt.Scan(&dataCari)

        var indexData int
        indexData = SequentialSearch(arrBuah, dataCari)

        if indexData > -1 {
            fmt.Printf("Data %s ditemukan pada indeks
ke-%d!", dataCari, indexData)
        } else if indexData == -1 {
            fmt.Printf("Data %s tidak ditemukan",
dataCari)
        }
    }
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Praktikum 2\Mi
Masukan data buah indeks ke - 0 : 20
Masukan data buah indeks ke - 1 : 23
Masukan data buah indeks ke - 2 : 25
Masukan data buah indeks ke - 3 : 27
Masukan data buah indeks ke - 4 : 28
Masukan data buah yang ingin dicari:
23
Data 23 ditemukan pada indeks ke-1!
PS D:\Semester 2\Alpro Praktikum 2>

```

2. Binary Search

```

package main

import "fmt"

const max = 100

type arr [max] int

func BinarySearch(a arr, n int, X int)bool{
    var kiri int = 0
    var kanan int = n - 1
    var found bool = false
    var tengah int

```

```

        for kiri <= kanan && !found{
            tengah = (kiri + kanan) / 2
            if a[tengah] < X {
                kiri = tengah + 1
            }else if a[tengah] > X {
                kanan = tengah - 1
            }else {
                found = true
            }
        }
        return found
    }

func main(){
    var data arr
    var n, X int

    fmt.Print("Input jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Input data ascending")
    for i:=0; i <= n; i++){
        fmt.Scanln(&data[i])
    }

    fmt.Println("Data yang dicari: ")
    fmt.Scan(&X)

    if BinarySearch(data, n, X) {
        fmt.Println("Data ditemukan")
    }else{
        fmt.Println("Data tidak ditemukan")
    }
}

```

Output :

```
PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Pra
Input jumlah data: 5
Input data ascending
10
20
30
40
50
Data yang dicari:
40
Data ditemukan
PS D:\Semester 2\Alpro Praktikum 2> █
```

3. Sequential Struct

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int)
int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim { //kurang dari
median, pencarian ke kiri
            kn = med - 1
        } else if nimDicari > arrData[med].nim { //lebih
dari median, pencarian ke kanan
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}
```

```

    }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int)
int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA
TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06 pada
indeks ke-%d ===\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari :
")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06,
nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada
array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
    }
}

```

```

        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan
pada array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch = binSearchStruct(mahasiswa06,
nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada
array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan
pada array!", nimDicari)
    }
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro
2\Minggu 11\Guided\SearchingStruct.go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : nehe
Masukkan NIM : 123
Masukkan IPK : 3.7
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : mia
Masukkan NIM : 345
Masukkan IPK : 4
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : pandu
Masukkan NIM : 246
Masukkan IPK : 3.6
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : indra
Masukkan NIM : 789
Masukkan IPK : 3.9
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : giri
Masukkan NIM : 357
Masukkan IPK : 3.2
-----

Masukkan NIM mahasiswa yang ingin dicari : 345

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 345 ditemukan pada array di indeks ke-1!
Berikut Detail Datanya :
Nama : mia
NIM : 345
IPK : 4.00

```

C. Unguided

1. Soal Unguided 1

- 1) Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

Jawaban :

```
package main

import "fmt"

type Calon struct {
    ID      int
    Suara int
}

func main() {
    var num int
    var suaraMasuk, suaraSah int
    var daftarCalon [20]Calon

    for i := 0; i < 20; i++ {
        daftarCalon[i].ID = i + 1
        daftarCalon[i].Suara = 0
    }

    for {
        _, err := fmt.Scan(&num)
        if err != nil || num == 0 {
            break
        }

        suaraMasuk++
    }
}
```

```

        if num >= 1 && num <= 20 {
            suaraSah++

            for i := 0; i < 20; i++ {
                if daftarCalon[i].ID == num {
                    daftarCalon[i].Suara++
                    break
                }
            }
        }

        fmt.Printf("Suara masuk: %d\n", suaraMasuk)
        fmt.Printf("Suara sah: %d\n", suaraSah)

        for i := 0; i < 20; i++ {
            if daftarCalon[i].Suara > 0 {
                fmt.Printf("%d: %d\n", daftarCalon[i].ID,
                    daftarCalon[i].Suara)
            }
        }
    }
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Praktikum 2\Minggu 1
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS D:\Semester 2\Alpro Praktikum 2>

```

Penjelasan :

Program di atas merupakan program yang saya buat untuk membaca, memvalidasi, dan menghitung perolehan suara dalam pemilihan ketua RT menggunakan metode sequential search. Di dalam program, saya membuat sebuah tipe data struct bernama Calon untuk menyimpan informasi ID dan jumlah suara masing-masing kandidat, serta menginisialisasi array bertipe struct tersebut sebanyak 20 elemen dengan nomor ID dari 1 sampai 20. Pada fungsi main, program menggunakan perulangan tak terbatas untuk menerima

input berupa nomor pilihan warga secara dinamis yang akan berhenti secara otomatis ketika membaca angka 0. Setiap data angka yang masuk akan dihitung ke dalam jumlah suara masuk, dan jika nomor tersebut valid berada di antara rentang 1 sampai 20, program akan menambah jumlah suara sah serta melakukan sequential search dengan menelusuri array satu per satu secara berurutan untuk mencocokkan ID calon lalu menambahkan jumlah suaranya. Setelah perulangan input berakhir, program akan menampilkan total suara masuk, total suara sah, serta melakukan perulangan akhir untuk mencetak daftar ID calon beserta jumlah perolehan suaranya khusus bagi calon yang mendapatkan suara lebih dari nol.

2. Soal Unguided 2

- 2) Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

No	Masukan	Keluaran
1	7 19 3 2 78 3 1 -3 18 19 0	Suara masuk: 10 Suara sah: 8 Ketua RT: 3 Wakil ketua: 19

Jawaban :

```
package main

import (
    "fmt"
)
```

```

type Calon struct {
    ID    int
    Suara int
}

func main() {
    var num int
    var suaraMasuk, suaraSah int
    var daftarCalon [20]Calon

    for i := 0; i < 20; i++ {
        daftarCalon[i].ID = i + 1
        daftarCalon[i].Suara = 0
    }

    for {
        _, err := fmt.Scan(&num)
        if err != nil || num == 0 {
            break
        }

        suaraMasuk++

        if num >= 1 && num <= 20 {
            suaraSah++

            for i := 0; i < 20; i++ {
                if daftarCalon[i].ID == num {
                    daftarCalon[i].Suara++
                    break
                }
            }
        }
    }

    fmt.Printf("Suara masuk: %d\n", suaraMasuk)
    fmt.Printf("Suara sah: %d\n", suaraSah)

    var ketua, wakil Calon

    for i := 0; i < 20; i++ {
        if daftarCalon[i].Suara > ketua.Suara {
            wakil = ketua
            ketua = daftarCalon[i]
        } else if daftarCalon[i].Suara > wakil.Suara {
            wakil = daftarCalon[i]
        }
    }

    fmt.Printf("Ketua RT: %d\n", ketua.ID)

```

```
        fmt.Printf("Wakil ketua: %d\n", wakil.ID)
    }
```

Output :

```
PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Pr
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS D:\Semester 2\Alpro Praktikum 2> █
```

Penjelasan :

Program di atas merupakan pengembangan dari program sebelumnya yang saya buat untuk menentukan pemenang ketua RT dan wakil ketua RT berdasarkan jumlah perolehan suara terbanyak pertama dan kedua. Struktur program untuk proses membaca input, menghitung total suara masuk, memvalidasi nomor pilihan, serta melakukan pembaruan data menggunakan pencarian *sequential search* masih mengimplementasikan logika yang sama dengan nomor satu. Perbedaannya terdapat setelah seluruh data suara selesai dibaca, di mana saya menambahkan logika penentuan pemenang dengan membuat dua variabel baru bertipe struct Calon untuk menampung data ketua dan wakil ketua terpilih. Program melakukan perulangan untuk mengecek seluruh data calon dan membandingkan jumlah suaranya; jika suara calon tersebut lebih besar dari suara ketua saat ini, maka posisi wakil akan mengambil alih data ketua lama dan posisi ketua diisi oleh data calon baru tersebut, sedangkan jika suaranya hanya lebih besar dari wakil, posisi wakil langsung diperbarui. Melalui kondisi perbandingan ini, apabila terdapat beberapa calon dengan jumlah suara sama, program secara otomatis mempertahankan calon dengan nomor urut terkecil karena proses penelusuran array dilakukan berurutan dari indeks awal, lalu di akhir program akan dicetak total suara masuk, suara sah, serta ID dari ketua RT dan wakil ketua RT yang baru.

3. Soal Unguided 3

- 3) Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan.

Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int
    fmt.Scan(&n, &k)

    isiArray(n)

    hasil := posisi(n, k)

    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kiri := 0
```

```

        kanan := n - 1

        for kiri <= kanan {
            tengah := (kiri + kanan) / 2

            if data[tengah] == k {
                return tengah
            } else if data[tengah] < k {
                kiri = tengah + 1
            } else {
                kanan = tengah - 1
            }
        }

        return -1
    }
}

```

Output :

```

PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Pra
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS D:\Semester 2\Alpro Praktikum 2> go run "d:\Semester 2\Alpro Pra
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS D:\Semester 2\Alpro Praktikum 2> █

```

Penjelasan :

Program di atas merupakan program yang saya buat untuk mencari posisi indeks dari suatu bilangan k di dalam kumpulan n data integer yang sudah terurut membesar menggunakan algoritma pencarian biner (*binary search*). Pada bagian awal, saya mendeklarasikan konstanta $NMAX$ bernilai satu juta dan array data secara global agar strukturnya dapat diakses secara langsung oleh seluruh subprogram. Di dalam fungsi main, program menerima input baris pertama berupa nilai n dan k , kemudian memanggil prosedur isiArray untuk membaca n buah data integer dari piranti masukan di baris kedua dan memasukkannya ke dalam array global menggunakan perulangan. Selanjutnya, program memanggil fungsi posisi untuk menerapkan metode *binary search* dengan menentukan batas penunjuk kiri dan kanan, lalu mencari posisi indeks

tengah secara berulang untuk mempersempit ruang pencarian menjadi setengahnya pada setiap tahap pengecekan nilai. Jika bilangan *k* ditemukan cocok dengan data di indeks tengah, fungsi akan mengembalikan nilai posisi indeks tersebut dengan perintah `return`, namun jika tidak ditemukan hingga batas pencarian habis, fungsi akan mengembalikan nilai minus satu. Pada akhir fungsi `main`, nilai kembalian dari fungsi posisi akan dievaluasi melalui kondisi pencabangan; jika bernilai minus satu maka program mencetak keluaran "TIDAK ADA", dan jika selain itu program akan mencetak nilai indeks posisi data tersebut ke layar.

D. Kesimpulan

Kesimpulan praktikum ini adalah saya berhasil memahami dan menerapkan algoritma *Sequential Search* dan *Binary Search* dalam bahasa Go. *Sequential Search* sangat cocok untuk data acak atau berjumlah sedikit karena memeriksa data satu per satu secara berurutan. Sebaliknya, *Binary Search* jauh lebih cepat dan efisien untuk menangani data skala besar, dengan syarat mutlak kumpulan data tersebut harus sudah dalam keadaan terurut.